

XML

Using XML

- 📖 Using the XML Data Type
- 📖 Retrieving XML by Using FOR XML
- 📖 Shredding XML by Using OPENXML
- 📖 Introducing XQuery
- 📖 Creating XML Indexes
- 📖 Implementing XML Schemas

Using the XML Data Type

- 📖 What Is XML?
- 📖 What Is the XML Data Type?
- 📖 The Query, Value, and Exist Methods
- 📖 The Modify Method
- 📖 The Nodes Method

What Is XML?

- XML is a plain-text, Unicode-based meta-language
- Represents both structured and semi-structured data
- Not tied to any programming language, OS, or vendor

A sample XML document for a music store order:

```
<?xml version="1.0" encoding="iso-8859-1" ?>
<?xml-stylesheet href="orders.xsl"?>

<order id="ord123456">
  <customer id="cust0921">
    <first-name>Dare</first-name>
    <last-name>Obasanjo</last-name>
    <address>
      <street>One Microsoft Way</street>
      <city>Redmond</city>
      <state>WA</state>
      <zip>98052</zip>
    </address>
  </customer>
</order>
```

What Is the XML Data Type?

- Native data type for XML
- Lets you store XML documents and fragments
- Used for tables, variables, or parameters
- Exposes methods to query and modify XML

-- usage within table definition

```
CREATE TABLE NewTable  
( Col1 int primary key,  
  Col2 xml )
```

-- usage as local variable

```
declare @data xml
```

-- usage as parameter to stored procedure

```
CREATE PROCEDURE SaveData(@doc xml) AS ...
```

The Query, Value, and Exist Methods

- Use query to return untyped XML
- Use value to return a scalar value
- Use exist to check existence of value
- Bind relational columns and variables

```
SELECT Invoices.query(  
  '<Store>  
    {sql:column("StoreName")}  
  </Store>')
```



The Modify Method

- Insert adds child nodes to an XML document
- Replace value of updates node in an XML doc
- Delete removes a node from the XML document

```
SET @xmlDoc.modify(  
    'delete  
    (/InvoiceList/Invoice/SalesPerson)[1]')
```



The Nodes Method

- Shreds XML variables into relational data
- Requires the APPLY operator with XML columns

```
SELECT nCol.value('../@OrderID[1]', 'int') ID,  
       nCol.value('@ProductID[1]', 'int') Prod  
FROM Orders  
CROSS APPLY OrderDoc.nodes('/Order/LineItem') AS nTable(nCol)
```



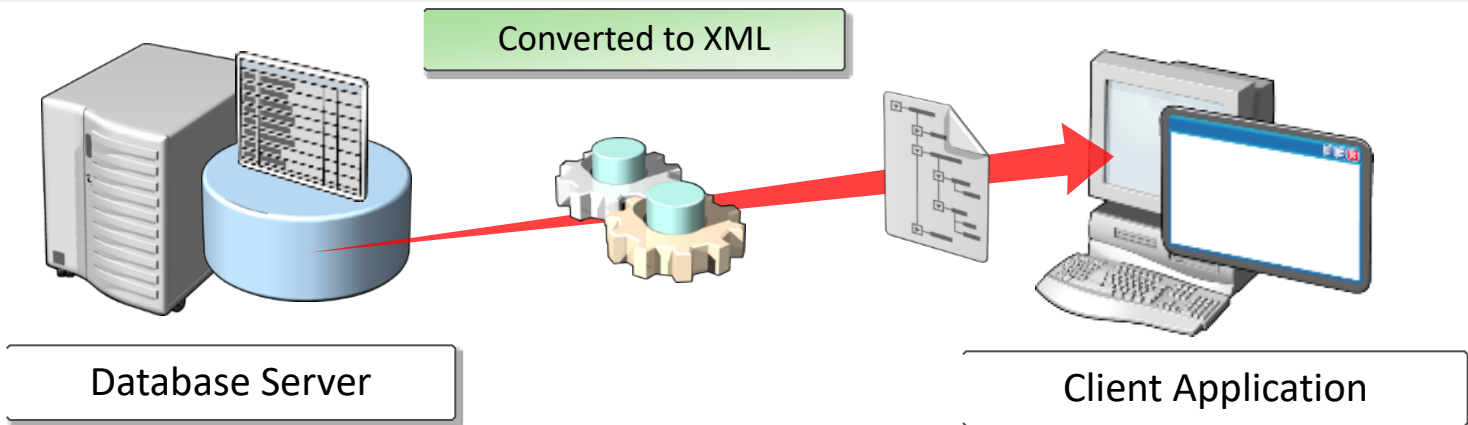
Retrieving XML by Using FOR XML

- 📖 Introduction to the FOR XML Clause
- 📖 What Are RAW Mode Queries?
- 📖 What Are AUTO Mode Queries?
- 📖 What Are EXPLICIT Mode Queries?
- 📖 What Are PATH Mode Queries?
- 📖 Syntax for Retrieving Nested XML

XML Demos

Introduction to the FOR XML Clause

- Extends SELECT syntax
- Returns XML instead of rows and columns
- Configurable to return attributes, elements, and schema
- Benefits client applications that work with XML



What Are RAW Mode Queries?

- XML representation of a rowset
- Contains either elements or attributes
- Optional root element and row element name

```
SELECT    Cust.CustomerID CustID, CustomerType, SalesOrderID
FROM      Customer Cust JOIN SalesOrderHeader [Order]
          ON Cust.CustomerID = [Order].CustomerID
ORDER BY  Cust.CustomerID
FOR XML RAW('Order'), ROOT('Orders')
```

```
<Orders>
  <Order><CustID>1</CustID><CustomerType>S</...
  </Order>
  ...
</Orders>
```



What Are AUTO Mode Queries?

XML representation of data entities

Execute this query:

```
SELECT Cust.CustomerID,  
       OrderHeader.CustomerID,  
       OrderHeader.SalesOrderID,  
       OrderHeader.Status,  
       Cust.CustomerType  
FROM Sales.Customer Cust, Sales.SalesOrderHeader OrderHeader  
WHERE Cust.CustomerID = OrderHeader.CustomerID  
ORDER BY Cust.CustomerID  
FOR XML AUTO
```

This is the partial result:

```
<Cust CustomerID="1" CustomerType="S">  
  <OrderHeader CustomerID="1" SalesOrderID="43860" Status="5" />  
  <OrderHeader CustomerID="1" SalesOrderID="44501" Status="5" />  
  <OrderHeader CustomerID="1" SalesOrderID="45283" Status="5" />  
  <OrderHeader CustomerID="1" SalesOrderID="46042" Status="5" />  
</Cust>
```

...

What Are EXPLICIT Mode Queries?

- Tabular representations of XML documents

Tag	Parent	Invoice!1!InvoiceNo	Invoice!1!Date!Element
1	NULL	43659	2001-07-01T00:00:00
1	NULL	43660	2001-07-02T00:00:00

- Allow complete control of XML format

```
SELECT 1 AS Tag, NULL AS Parent,  
       SalesOrderID AS [Invoice!1!InvoiceNo],  
       OrderDate AS [Invoice!1!Date!Element]  
FROM SalesOrderHeader  
FOR XML EXPLICIT
```

```
<Invoice InvoiceNo="43659">  
  <Date>2001-07-01T00:00:00</Date>  
</Invoice>  
<Invoice InvoiceNo="43660">...
```

Attribute

Element



What Are PATH Mode Queries?

- Use XML Path Language (XPath) to specify XML format
- Allow creation of nested data
- Easier to use than EXPLICIT mode

```
SELECT    EmployeeID "@EmpID",
          FirstName "EmpName/First",
          LastName  "EmpName/Last"
FROM      Person.Contact INNER JOIN Employee
          ON Person.Contact.ContactID = Employee.ContactID
FOR XML PATH
```

```
<row EmpID="1">
  <EmpName>
    <First>Guy</First>
    <Last>Gilbert</Last>
  </EmpName>
</row>
```

...

Syntax for Retrieving Nested XML

- AUTO mode produces only attributes or elements
- Use inner FOR XML with TYPE clause to return xml data type
- Combine EXPLICIT mode with UNION ALL

```
SELECT 1 AS Tag, NULL AS Parent, ...  
FROM SalesOrderHeader  
UNION ALL  
SELECT 2 AS Tag, 1 AS Parent, ...  
FROM SalesOrderDetail OD JOIN ...  
FOR XML EXPLICIT
```

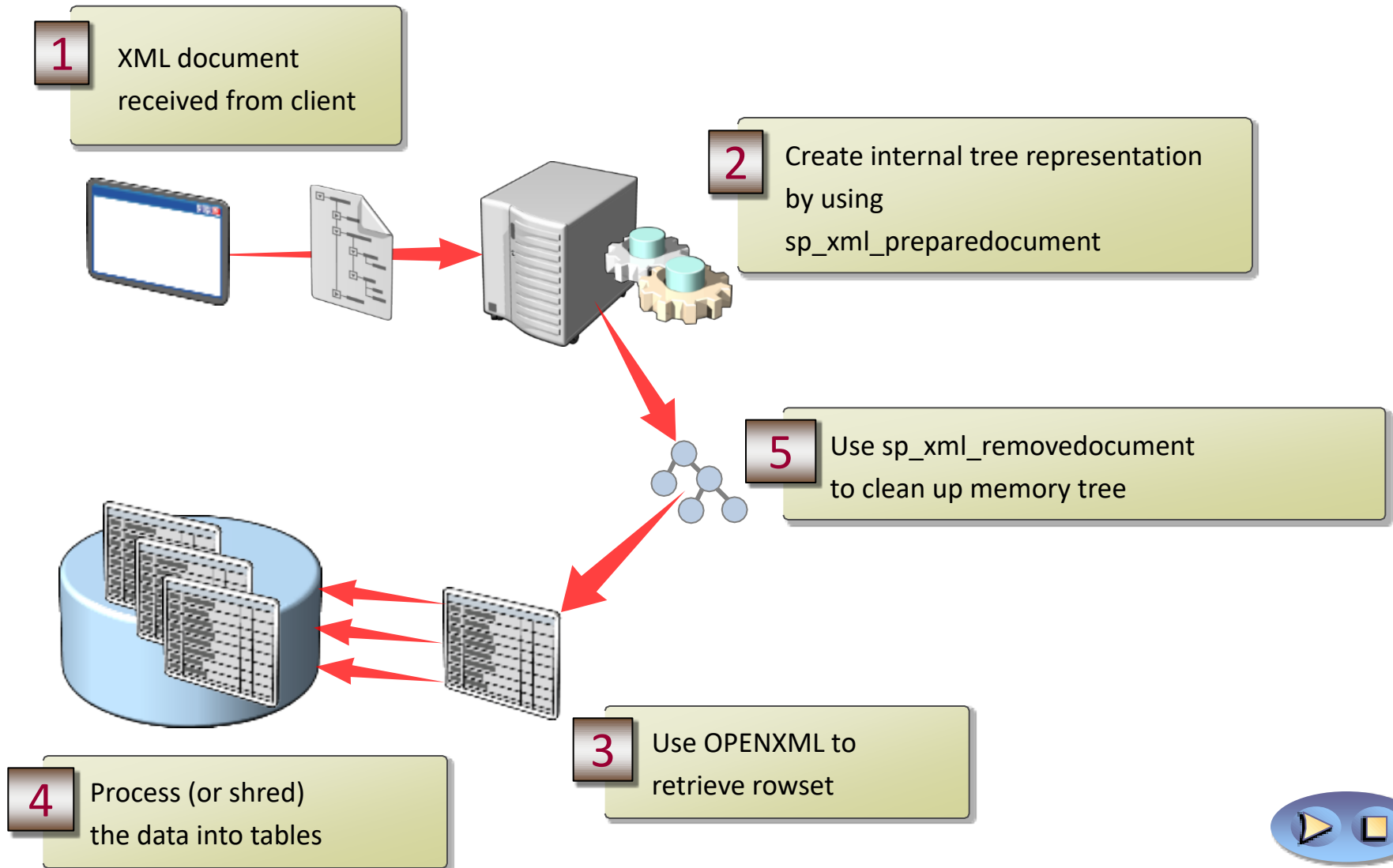
```
<Invoice InvoiceNo="43659">  
  <Date>2001-07-01T00:00:00</Date>  
  <LineItem ProductID="709">Bike Socks, M</LineItem>  
  <LineItem ProductID="711">Helmet, Blue</LineItem>  
</Invoice>...
```



Shredding XML by Using OPENXML

- 📖 Overview of Shredding XML Data
- 📖 Stored Procedures for Managing In-Memory Node Trees
- 📖 OPENXML Syntax
- 📖 Syntax for Working with XML Namespaces

Overview of Shredding XML Data



Stored Procedures for Managing In-Memory Node Trees

- Create tree by using `sp_xml_preparedocument`
- Free memory by using `sp_xml_removedocument`

```
CREATE PROC ProcessOrder  
@doc xml -- xml data
```

```
AS
```

```
-- Declare document handle  
DECLARE @hdoc integer
```

```
-- Create memory tree  
EXEC sp_xml_preparedocument @hdoc OUTPUT, @doc
```

```
-- Process Document
```

```
-- Remove memory tree  
EXEC sp_xml_removedocument @hdoc
```

OPENXML Syntax

ColPattern identifies SalesOrderID, Status, and OrderDate

```
<Customer CustomerID="1" CustomerType="S">
  <Order SalesOrderID="43860" Status="5"
    OrderDate="2001-08-01T00:00:00">
    <OrderDetail ProductID="761" Quantity="2"/>
    <OrderDetail ProductID="770" Quantity="1"/>
  </Order>
</Customer>
```

rowpattern identifies node level

Uses attributes as default

```
SELECT *
FROM OPENXML (@idoc, '/Customer/Order/OrderDetail', 1)
WITH (CustomerID int      '..../@CustomerID',
      OrderID  int      '..../@SalesOrderID',
      OrderDate datetime  '..../@OrderDate',
      ProdID   int      '@ProductID',
      Quantity int)
```

From
Customer
element

From Order element

Defaults to Quantity attribute

From OrderDetail element



Syntax for Working With XML Namespaces

- sp_xml_preparedocument accepts namespaces
- Use namespace prefix in all XPath expressions

```
<Customer xmlns="urn:AW_NS" xmlns:o="urn:AW_OrderNS"
  CustomerID="1" CustomerType="S">
  <o:Order SalesOrderID="43860" Status="5"
    OrderDate="2001-08-01T00:00:00">
    <o:OrderDetail ProductID="761" Quantity="2"/>
    <o:OrderDetail ProductID="770" Quantity="1"/>
  </o:Order>
</Customer>
```

```
EXEC sp_xml_preparedocument @idoc OUTPUT, @doc,
<ROOT xmlns:rootNS="urn:AW_NS" xmlns:orderNS="urn:AW_OrderNS"/>
```

```
SELECT * FROM OPENXML (@idoc,
'/rootNS:Customer/orderNS:Order/orderNS:OrderDetail')
```

...

Introducing XQuery

- 📖 What Is XQuery?
- 📖 XQuery Basics
- 📖 XQuery Expressions

What Is XQuery?

- Query language to identify nodes in XML
- Can query structured or semi-structured XML

```
/InvoiceList/Invoice[@InvoiceNo=1000]
```

- FLWOR statements

Statement	Description
For	Iterate through sibling nodes
where	Apply filtering criteria to the iteration
order by	Sort values in returned resultset
return	Specify the XML to be returned

XQuery Basics

Sequences and QNames

- Result of an XQuery expression is a sequence
- All identifiers are QNames

Operators

- Arithmetic comparison
- General comparison
- Value comparison
- Node comparison
- Node order comparison
- Logical

if-then-else

```
if ( $A eq $B )  
then <result>A</result>  
else <result>B</result>
```

Comments

```
(: Comment text :)
```

XQuery Expressions

Primary expressions

- Literals
- Variable references
- Function calls

Path expressions

- Relative

child::Address/child::Country

- Absolute

/Address/Country

Sequence expressions

- Construct, filter and combine sequences

```
declare @x xml
set @x = '<root>
<abc></abc>
<abc attrAbc="1"></abc>
<abc attrAbc="2"></abc>
</root>'
SELECT @x.query('/root/abc[attrAbc]')
```


STORED PROCEDURES



Execution Environment

Code can be executed at two locations

-  Client

-  Server

Advantages of client-side programming

-  Can more easily serve users with vastly different needs

-  Can avoid additional network traffic

-  Can distribute processor load to local environments

-  Can create better user-interface interaction

Advantages of server-side programming

-  Can more easily ensure uniform application of business rules

-  Can update code more easily

What Are Stored Procedures?

A collection of T-SQL statements stored on the server

Stored Procedures Can:

- Accept input parameters
- Return output parameters or rowset
- Return a status value to indicate success or failure

Benefits of using Stored Procedures:

- Promotes modular programming
- Provides security attributes and permission chaining
- Allows delayed binding and code reuse
- Reduces network traffic

What Are Stored Procedures?

 Allows the end user to execute a group of precompiled statements by entering a single predefined command

 --benefits

 --Simplify repeated tasks

 --Run faster

 --Reduce network traffic

 --Can catch user errors before they are entered into the database

 --Establish consistency by performing tasks the same way

 --Help to provide security

 --Can enforce complex business rules and defaults

Types of Stored Procedures

- 📖 User-defined stored procedures
 - 📋 User-defined procedures that must be explicitly called
- 📖 Triggers
 - 📋 User-defined procedures that execute automatically when data in a given table is modified
- 📖 System procedures
 - 📋 Built in procedures that read or modify one or more system tables

Performance Benefits

A compiled stored procedure executes more rapidly than a batch because:

- 📖 The stored procedure has already been parsed
- 📖 Object references in the stored procedure have already been resolved
- 📖 Adaptive Server does not need to build a query tree; it can use the one in *sysprocedures*
- 📖 If an unused query plan for the procedure exists in procedure cache, Adaptive Server does not need to create a query plan

Guidelines for Creating Stored Procedures

Rules for designing stored procedures:

- ü Qualifying names inside of stored procedures

- ü Obfuscating procedure definitions

- ü SET statement options

- ü Naming conventions

- ü Execution Context

- ü Using @@nestlevel

Syntax for Creating Stored Procedures

```
CREATE { PROC | PROCEDURE } [schema_name.] procedure_name [ ; number ]
    [ { @parameter [ type_schema_name. ] data_type }
      [ VARYING ] [ = default ] [ OUT | OUTPUT ] [ READONLY ]
    ] [ ,...n ]
[ WITH <procedure_option> [ ,...n ] ]
[ FOR REPLICATION ]
AS { <sql_statement> [;][ ...n ] | <method_specifier> }
[;]
<procedure_option> ::=
    [ ENCRYPTION ]
    [ RECOMPILE ]
    [ EXECUTE_AS_Clause ]

<sql_statement> ::=
{ [ BEGIN ] statements [ END ] }
<method_specifier> ::=
EXTERNAL NAME assembly_name.class_name.method_name
```


Syntax for Altering Stored Procedures

```
ALTER { PROC | PROCEDURE } [schema_name.] procedure_name [ ; number ]  
    [ { @parameter [ type_schema_name. ] data_type }  
      [ VARYING ] [ = default ] [ [ OUT [ PUT ] ] [ ,...n ] ]  
    [ WITH <procedure_option> [ ,...n ] ]  
    [ FOR REPLICATION ]  
    AS
```

```
        { <sql_statement> [ ...n ] | <method_specifier> }
```

```
<procedure_option> ::=  
[ ENCRYPTION ]  
[ RECOMPILE ]  
[ EXECUTE_AS_Clause ]
```

```
<sql_statement> ::=  
{ [ BEGIN ] statements [ END ] }
```

```
<method_specifier> ::=  
EXTERNAL NAME  
assembly_name.class_name.method_name
```

Syntax for Dropping Stored Procedures

- `DROP { PROC | PROCEDURE } { [ schema_name. ] procedure } [ ,...n ]`

Removes one or more stored procedures or procedure groups from the current database.

How Are Stored Procedures Created?

Creating a Stored Procedure

```
CREATE PROCEDURE HumanResources.usp_GetEmployeesName
@NamePrefix char(1)
AS
BEGIN
SELECT BusinessEntityID, FirstName, LastName, EmailAddress
FROM HumanResources.vEmployee
WHERE FirstName LIKE @NamePrefix + '%'
ORDER BY FirstName
END
```

Calling a Stored Procedure

```
EXECUTE HumanResources.usp_GetEmployeesName 'A'
```

Stored Procedures and Permissions

📖 To allow other people to use your stored procedure, you must grant them permission

📖 Simplified syntax:

```
grant execute  
on procedure_name  
to user_list
```

📖 Example:

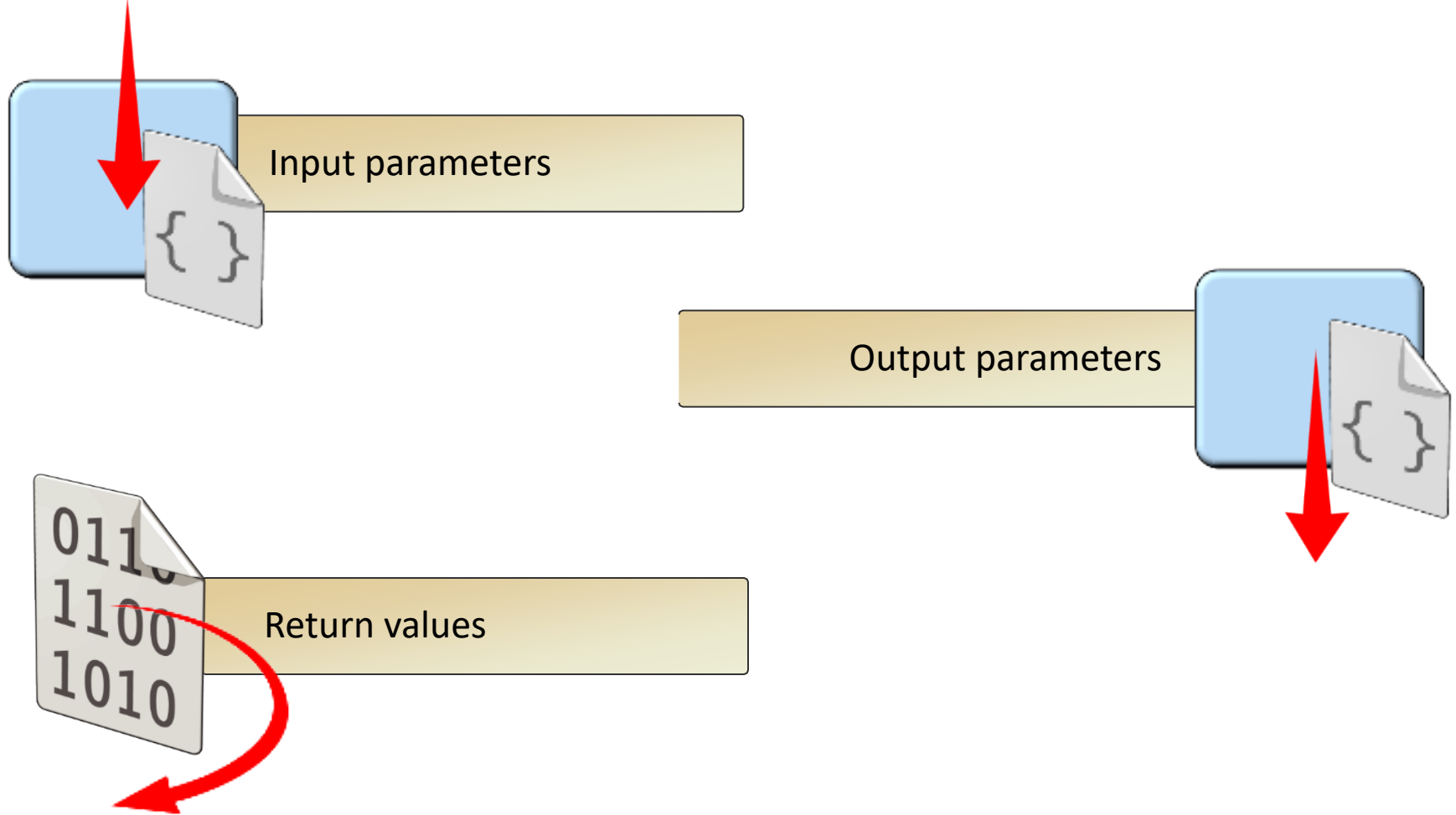
```
grant execute  
on proc_update_titles  
to tjovanka, vturlough, pbrown
```

Creating Parameterized Stored Procedures

-  Stored Procedure Parameters
-  Table-valued Parameters

Stored Procedure Parameters

Parameterized stored procedures contain 3 major components:



Passing Parameters

- 📖 Two methods for passing values to parameters:
 - 📋 Passing by parameter position
 - 📋 Passing by parameter name

Default Value

📖 A default value is a value assigned to a parameter for which no value has been received from the **exec** statement

📖 Example:

```
create proc proc_state authors
    (@state char(2) = "CA")
as
    select au_lname, au_fname, state
    from authors
    where state = @state
return
```

```
exec proc_state_authors
```

```
-- No state value passed
```

au_lname	au_fname	state
-----	-----	-----
White	Johnson	CA
Green	Marjorie	CA
...		

Input Parameters: Common Errors

- ❏ Passed parameters are not compatible with the datatype of the parameter
- ❏ Within the same statement, a parameter is passed by position after a parameter has been passed by name
 - 📋 Although it is not recommended, it is possible to mix the two methods of passing; however, after one parameter is passed by name, all subsequent parameters in that statement must be passed by name
- ❏ One or more parameters is missing
 - 📋 Missing parameters can be overcome by using default values for input parameters
- ❏ Parameters are passed by position in the wrong order

Table-valued Parameters

Table-valued Parameters:



Are new in SQL 2008



Can be passed to stored procedures



Pass multiple rows of data to a stored procedures

System Procedures for Stored Procedures

- 📖 **sp_depends** {*table_name* / *procedure_name*}
 - 📋 When given a table, lists all objects (including procedures) in the same database that reference that table
 - 📋 When given a procedure, lists all tables in the same database referenced by the procedure
- 📖 **sp_help** *procedure_name*
 - 📋 Displays information about the specified procedure
- 📖 **sp_helptext** *procedure_name*
 - 📋 Displays the text used to create the specified procedure
- 📖 **sp_rename** *old_proc_name, new_proc_name*
 - 📋 Changes the name of a procedure

Valid and Invalid Statements

 A stored procedure can:

-  Select and modify data
-  Create temporary and permanent tables
-  Call other stored procedures
-  Reference objects in its own database and in other databases

 A stored procedure cannot execute these statements:

-  **use *database***
-  **create view**
-  **create default**
-  **create rule**
-  **create procedure**
-  **create trigger**

Server Cursor

 A server cursor is a cursor created in a stored procedure

 Example:

```
create proc proc_fetch_book
as
declare      @title          char(30) ,
              @title_id      char(6)
declare biz_book cursor
    for select title, title_id from titles
        where type = "business"
    for read only
open biz_book
fetch biz_book into @title, @title_id
-- additional processing here
close biz_book
deallocate cursor biz_book
return
```

Stored Procedure Best Practices

Stored Procedure Tips

- Use WITH ENCRYPTION to hide procedure source
- Use WITH RECOMPILE to force recompilation on each execution

Stored Procedure Best Practices

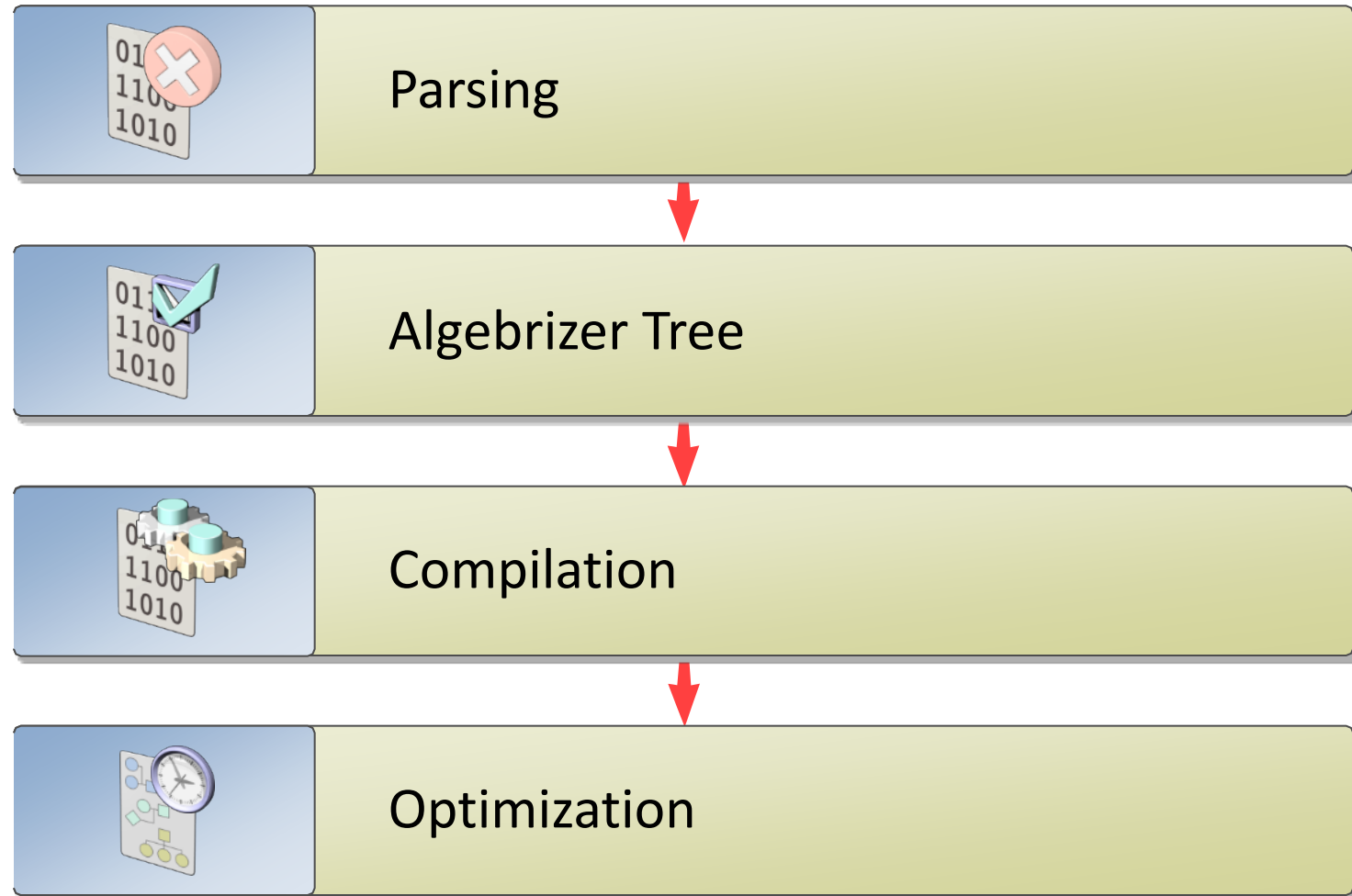
- Validate all input parameters
- Avoid building string based SQL within procedure to reduce the risk of SQL injection
- Use cursors sparingly

Working With Execution Plans



- What Is an Execution Plan?
- Viewing an Execution Plan
- Execution Plan Caching
- Query Compilation
- Forced Stored Procedure Recompilation

Query Compilation



Forced Stored Procedure Recompilation

Stored procedure recompilation

- 1 `sp_recompile`
- 2 `WITH RECOMPILE` at creation
- 3 `WITH RECOMPILE` on execution

```
USE AdventureWorks2008;  
GO  
EXEC sp_recompile N'Sales.Customer';  
GO
```

Handling Exceptions

- 📖 Syntax for Structured Exception Handling
- 📖 Guidelines for Handling Exceptions

Syntax for Structured Exception Handling

- TRY...CATCH blocks provide the structure
 - § TRY block contains protected transactions
 - § CATCH block handles exceptions

```
BEGIN TRY
    SELECT *
    FROM sys.messages
    WHERE message_id = 21;
END TRY

BEGIN CATCH
    SELECT ERROR_NUMBER() AS ErrorNumber;
END CATCH;
GO
```

Guidelines for Handling Exceptions

TRY/CATCH requirements:

Each TRY...CATCH construct must be inside a single batch

ü

stored procedure, or trigger

A TRY block must be immediately followed by a CATCH

ü

block

ü

TRY...CATCH constructs can be nested

Questions

- 📖 In what order must you supply parameters in the form **@parameter** = *value*?
- 📖 What should you do if you have a slow performing query?

Implementing Triggers

- 📖 What Are Triggers?
- 📖 How an INSERT Trigger Works
- 📖 How a DELETE Trigger Works
- 📖 How an UPDATE Trigger Works
- 📖 How an INSTEAD OF Trigger Works
- 📖 How Nested Triggers Work
- 📖 Considerations for Recursive Triggers

What Are Triggers?

Triggers are:

- Special stored procedures that execute when INSERT, UPDATE, or DELETE statements modify a table
- Part of a single transaction along with the initiating statement

Two categories:

- AFTER triggers execute after an INSERT, UPDATE, or DELETE statement
- INSTEAD OF triggers execute instead of an INSERT, UPDATE, or DELETE statement

Typical Trigger Applications

- 📖 Cascading modifications through related tables
- 📖 Rolling back changes that violate data integrity
- 📖 Enforcing restrictions that are too complex for rules or constraints
- 📖 Maintaining duplicate data
- 📖 Maintaining columns with derived data
- 📖 Performing custom recording

Rules for Triggers

Triggers can:

-  Declare local variables
-  Invoke nontrigger stored procedures

Triggers cannot:

-  Be called directly
-  Use parameters
-  Be defined on temporary tables or views
-  Create permanent database objects

Minimally logged operations (such as **truncate table**) do not cause triggers to fire

Trigger

- 📖 A trigger is a stored procedure created on a given table that executes automatically when data in the table is modified

	Can It Be Explicitly Called?	Can It Be Executed Automatically?	Can It Use Parameters?
User-Defined Stored Procedure	Yes	No	Yes
Trigger	No	Yes	No

Creating Triggers



Simplified syntax:

```
create trigger trigger_name  
on table_name  
for {insert | update | delete} [, {insert | update | delete} ...]  
as  
    sql_statements
```

How an INSERT Trigger Works

- 1 INSERT statement executed
- 2 INSERT statement logged
- 3 AFTER INSERT trigger statements executed

```
create trigger trg_i_sales
on sales
for insert
as
SET NOCOUNT ON;
if datename (dw,getdate()) = "Sunday"
begin
    raiserror 40070, "Sales cannot be
        processed on Sunday."
    rollback tran
    return
end
```

How a DELETE Trigger Works

1 DELETE statement executed

2 DELETE statement logged

3 AFTER DELETE trigger statements executed

```
CREATE TRIGGER [delCategory] ON [Categories]
AFTER DELETE AS
BEGIN
    UPDATE P SET [Discontinued] = 1
    FROM [Products] P INNER JOIN deleted as d
    ON
    P.[CategoryID] = d.[CategoryID]
END;
```

How an UPDATE Trigger Works

- 1 UPDATE statement executed
- 2 UPDATE statement logged
- 3 AFTER UPDATE trigger statements executed

```
CREATE TRIGGER [updtProductReview] ON [Production].[ProductReview]
AFTER UPDATE NOT FOR REPLICATION AS
BEGIN
    UPDATE [Production].[ProductReview]
    SET [Production].[ProductReview].[ModifiedDate] =
        GETDATE() FROM inserted
    WHERE inserted.[ProductReviewID] =
        [Production].[ProductReview].[ProductReviewID];
END;
```

How an INSTEAD OF Trigger Works

1 UPDATE, INSERT, or DELETE statement executed

2 Executed statement does not occur

3 INSTEAD OF trigger statements executed

```
CREATE TRIGGER [delEmployee] ON [HumanResources].[Employee]
  INSTEAD OF DELETE NOT FOR REPLICATION AS
  BEGIN
    SET NOCOUNT ON;
    DECLARE @DeleteCount int;
    SELECT @DeleteCount = COUNT(*) FROM deleted;
    IF @DeleteCount > 0
    BEGIN ...
    END;
  END;
```

Dropping Triggers

 Simplified syntax:

```
drop trigger trigger_name
```

 Example:

```
drop trigger trg_i_sales
```


Disabling Triggers

- 📖 As previously stated, **insert**, **update**, and **delete** statements cause triggers defined on a table to fire
 - 📋 This can reduce performance unnecessarily when loading large amounts of data into a table
- 📖 You can use the **alter table** command to disable triggers on a table, significantly enhancing database loading time
 - 📋 Because triggers are often used to enforce integrity constraints, you should disable triggers with caution
 - 📋 Triggers must be explicitly re-enabled

disable trigger Syntax and Example

Simplified syntax:

```
alter table table_name  
{enable | disable} trigger  
trigger_name [, ...]
```

```
alter table table_name  
{enable | disable} trigger
```

Example:

```
alter table sales  
disable trigger  
trg_i_sales
```

System Procedures for Triggers

sp_depends {*table_name* / *trigger_name*}

-  When given a table, lists all objects (including triggers) in the same database that reference that table
-  When given a trigger, lists all tables in the same database referenced by the trigger

sp_help *trigger_name*

-  Displays information about the specified trigger

sp_helptext *trigger_name*

-  Displays the text used to create the specified trigger

sp_rename *old_trigger_name*, *new_trigger_name*

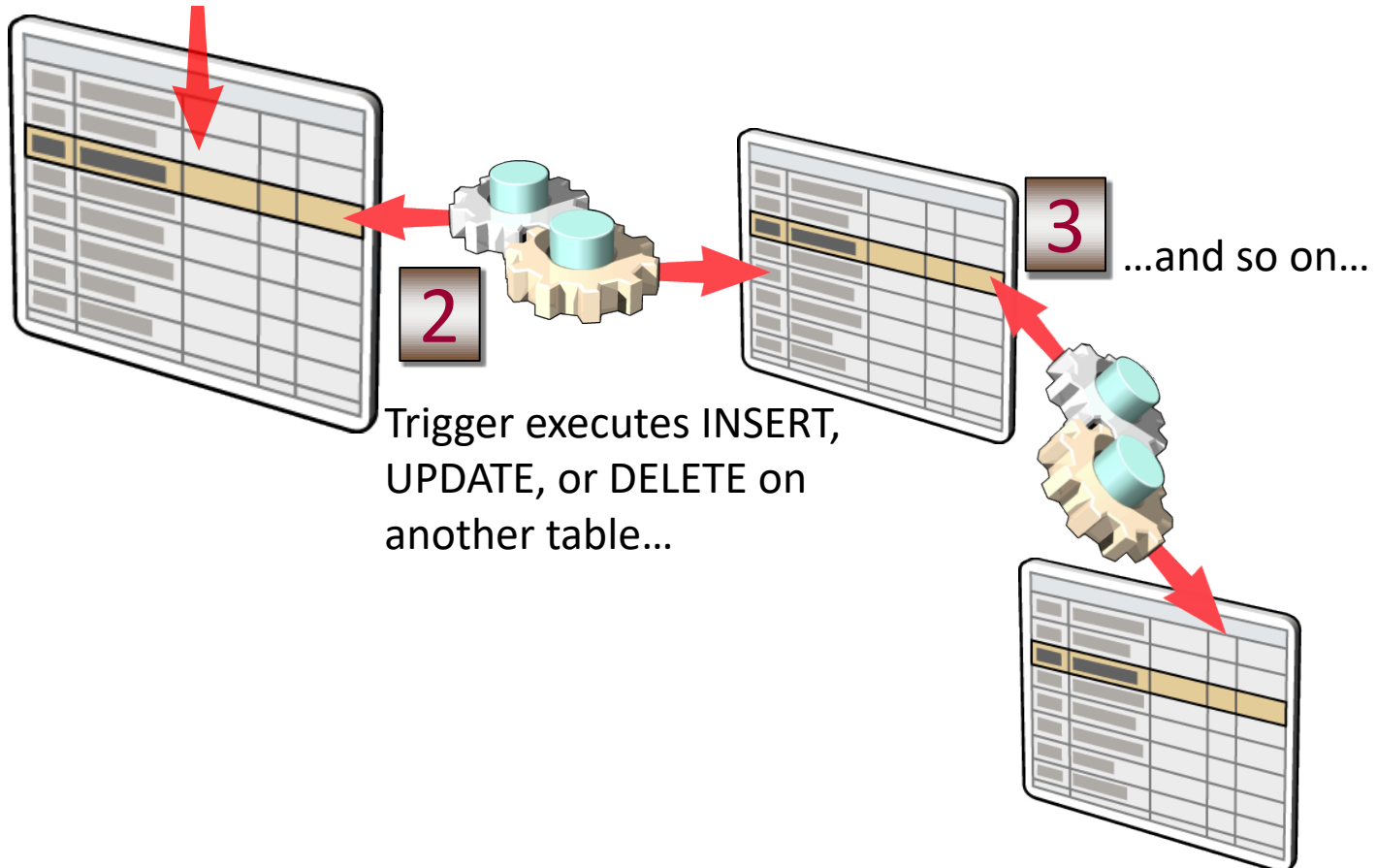
-  Changes the name of a trigger

How Triggers Work

- Triggers can roll back transactions if a specific business rule is not satisfied
- When a trigger that contains a rollback statement is executed from an SQL batch, the entire batch is canceled
- Any statement following the ROLLBACK TRANSACTION statement will still be executed
- Any modifications that happen after the rollback are not rolled back

How Nested Triggers Work

1 INSERT, UPDATE, or DELETE statement



The *inserted* and *deleted* Tables

- 📖 *inserted* and *deleted* are two tables that are automatically created for a trigger whenever it fires
 - 📋 *inserted* records any rows added to the table
 - 📋 *deleted* records any rows removed from the table

Rules for *inserted* and *deleted* Tables

- 📖 Both tables have the same columns as the trigger table
- 📖 The trigger can select data from the two tables
 - 📋 Other processes cannot select data from the two tables
- 📖 The trigger cannot modify data in the two tables
- 📖 Each nested trigger has its own *inserted* and *deleted* table
 - 📋 If a trigger modifies data in its trigger table, the changes are not reflected in that trigger's *inserted* and *deleted* tables

INSERT and the OUTPUT Clause

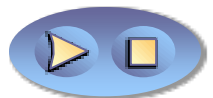
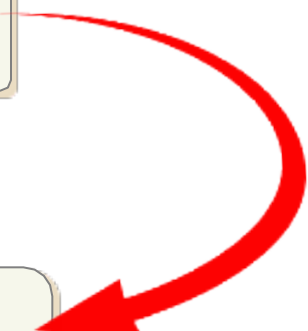
- Using OUTPUT in an INSERT statement returns information from each row affected by the INSERT statement

Syntax:

```
INSERT SomeTable  
OUTPUT dml_select_list INTO  
(@table_variable | output_table) (column_list)
```

Example Query:

```
DECLARE @MyTableVar table( ScrapReasonID smallint,  
                           Name varchar(50),  
                           ModifiedDate datetime);  
INSERT Production.ScrapReason  
  OUTPUT INSERTED.ScrapReasonID, INSERTED.Name,  
  INSERTED.ModifiedDate  
  INTO @MyTableVar  
VALUES (N'Operator error', GETDATE());
```



DELETE and the OUTPUT Clause

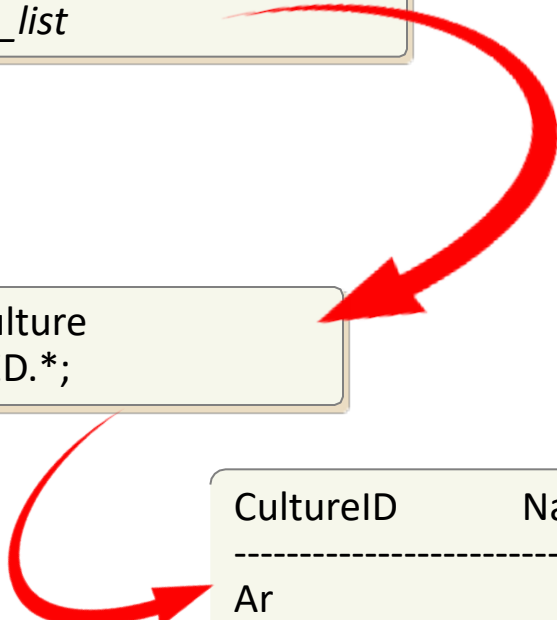
- Using OUTPUT in a DELETE statement removes a row from a table and returns the deleted values to a result set

Syntax:

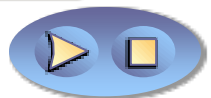
```
DELETE SomeTable  
    OUTPUT column_list
```

Example Query:

```
DELETE Production.Culture  
    OUTPUT DELETED.*;
```



CultureID	Name	ModifiedDate
Ar	Arabic	1998-06-01
En	English	1998-06-01



UPDATE and the OUTPUT Clause

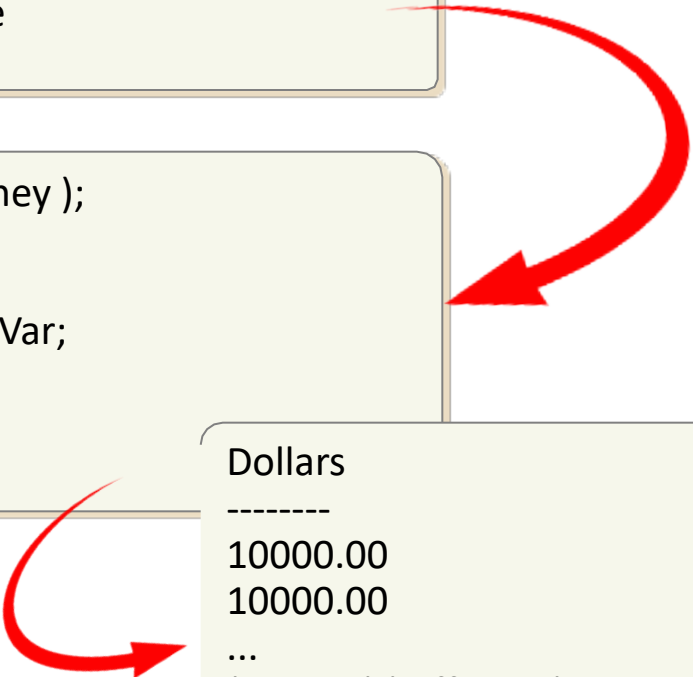
- Using OUTPUT in an UPDATE statement returns information from each row affected by the UPDATE statement

Syntax:

```
UPDATE SomeTable  
OUTPUT dml_select_list FROM table_source  
WHERE search_condition
```

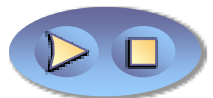
```
DECLARE @NewTableVar table ( Dollars money );  
UPDATE Sales.SalesPerson  
SET Bonus = 10000  
OUTPUT INSERTED.Bonus INTO @NewTableVar;
```

```
SELECT Dollars  
FROM @NewTableVar;
```



A red curved arrow originates from the 'OUTPUT' clause in the first SQL block and points to the 'OUTPUT' clause in the second SQL block. Another red curved arrow originates from the 'SELECT' statement in the third SQL block and points to the output result box.

```
Dollars  
-----  
10000.00  
10000.00  
...  
(17 row(s) affected)
```



Trigger Production Practices

📖 The following issues should be considered when developing effective triggers:

📋 @@rowcount

📋 **if update**

📋 Nested triggers

📋 Recursive triggers

Review of @@rowcount

- 📖 @@rowcount is a global variable that returns the number of rows affected by the previous statement
- 📖 Useful for determining if the statement causing a trigger to fire affected 0, 1, or multiple rows
- 📖 Most statements affect the value of @@rowcount
 - 📋 If you need the value of @@rowcount at a time other than the beginning of the trigger, you must declare a local variable and store the value of @@rowcount in that variable

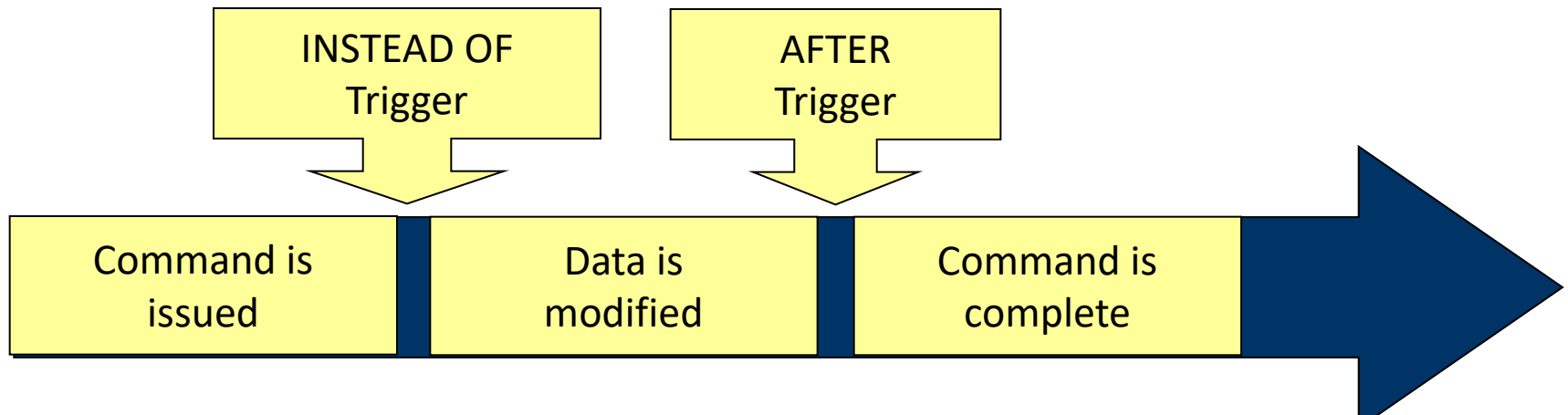
if update

- 📖 **if update** is a condition that allows a trigger to check for a change in a specific column
- 📖 It can be used only with triggers
- 📖 Usually used to check if values in a primary key column have changed
- 📖 Simplified syntax:
`if update (column_name) [ {and | or} update (column_name)]...`

Nested Trigger

- 📖 A nested trigger is a trigger that fires in response to modifications executed within a trigger
- 📖 The maximum level of nesting is 16
 - 📋 Both basic stored procedures and triggers count towards the maximum level
 - 📋 `@@nestlevel` is a global variable that returns the current nesting level

Trigger Types and Limitations



Trigger Limitations

- A trigger can apply to only one table
- Triggers are executed only in the current database
- Triggers must belong to the same schema as their target table
- INSTEAD OF DELETE/UPDATE triggers cannot be created on a table that has a cascading foreign key defined.

Triggers and Transactions

- 📖 A trigger is part of the transaction that causes it to fire
- 📖 The trigger can roll back:
 - 📋 The entire transaction
 - 📋 Itself only
 - 📋 Itself and the statement that caused it to fire

Rolling Back a Transaction

Case A

Stored procedure

```
begin tran
...
insert ...
print "in sp"
...
commit tran
print "sp done"
```

Trigger

```
....
....
rollback tran
print "tr done"
return
```

Case B

```
begin tran
...
insert ...
print "in sp"
...
commit tran
print "sp done"
```

Trigger

```
begin tran
....
rollback tran
print "tr done"
return
```

Rolling Back a Trigger

Case C

Stored procedure

```
begin tran
...
insert ...
print "in sp"
...
commit tran
print "sp done"
```

Trigger

```
save tran s1
....
rollback tran s1
print "tr done"
return
```

Case D

(this case `insert`
causes an error)

```
begin tran
...
insert ...
print "in sp"
...
commit tran
print "sp done"
```

Trigger

```
begin tran s2
....
rollback tran s2
print "tr done"
return
```

rollback trigger

 **rollback trigger** rolls back a trigger and the statement that fired the trigger

 **Syntax:**

`rollback trigger [with raiserror error_number [error_statement] ]`

 **Example:**

```
create trigger trg_i_publishers
on publishers
for insert
as
    if @@rowcount > 1
        begin
            rollback trigger with raiserror 40031
            "You cannot insert more than one
            publisher at a time."
        return
    end
```

rollback trigger: Example

Case E

Stored procedure

```
begin tran
...
insert ...
print "in sp"
...
commit tran
print "sp done"
```

Trigger

```
....
....
rollback trigger
print "tr done"
return
```

Updating Key Values

Desired Action	Constraints	Triggers
Insert a primary key value	Allowed	Allowed
Insert a foreign key value	Allowed	Allowed
Update a primary key value	Not allowed*	Allowed
Update a foreign key value	Allowed	Allowed
Delete a primary key value	Not allowed*	Allowed
Delete a foreign key value	Allowed	Allowed

*Primary key values can be updated or deleted if they are not referenced in any foreign keys.

- 📖 Primary key deletions or updates are possible only through triggers
- 📖 Cascaded changes are possible only through triggers

Constraints versus Triggers

Advantages to constraints:

-  Constraints (and rules) are faster than triggers
-  Constraints do not require additional coding
-  It is better to check data before it is entered into the database than to remove it after it is already there

Advantages to triggers:

-  Flexible
 - Triggers can do anything you can code
-  Best for business rules that cannot be expressed as referential constraints, such as cascaded updates or deletes

How to Write Linked Server-Based Distributed Queries

- Use a fully qualified four-part name and the OPENQUERY function
- The OPENQUERY function executes the query on the specified linked server

```
SELECT *  
FROM OPENQUERY(OracleSvr, 'SELECT name, id FROM  
HumanResources.Titles')
```

Questions

- 📖 What is the difference between scalar and table type user-defined functions?
- 📖 Why would a linked server be used instead of an ad hoc distributed query?
- 📖 How is a table type user-defined function queried?
- 📖 Where are stored procedure execution plans stored after initial execution?

- ***Cursor***
- ***Administration Tasks***
- ***SQL Server Security Model***
 - ***Principal***
 - ***Securable***
 - ***Permission***
 - § ***Creating users***
 - ***Backup***
 - ***Restore***
 - ***Job & alerts and operators***

• ***SQLCLR***

Additional Topics

Data base mirroring

Linked servers

Service broker

Notification service

**Query tuning and
performance**

Securing SQL server and
DB encryption

Partitioning

Full-text search

SQL injection

Replication

SQL Debugging

Cursor

- 📖 A cursor is a mechanism that processes the result set of a predefined query on a row-by-row basis

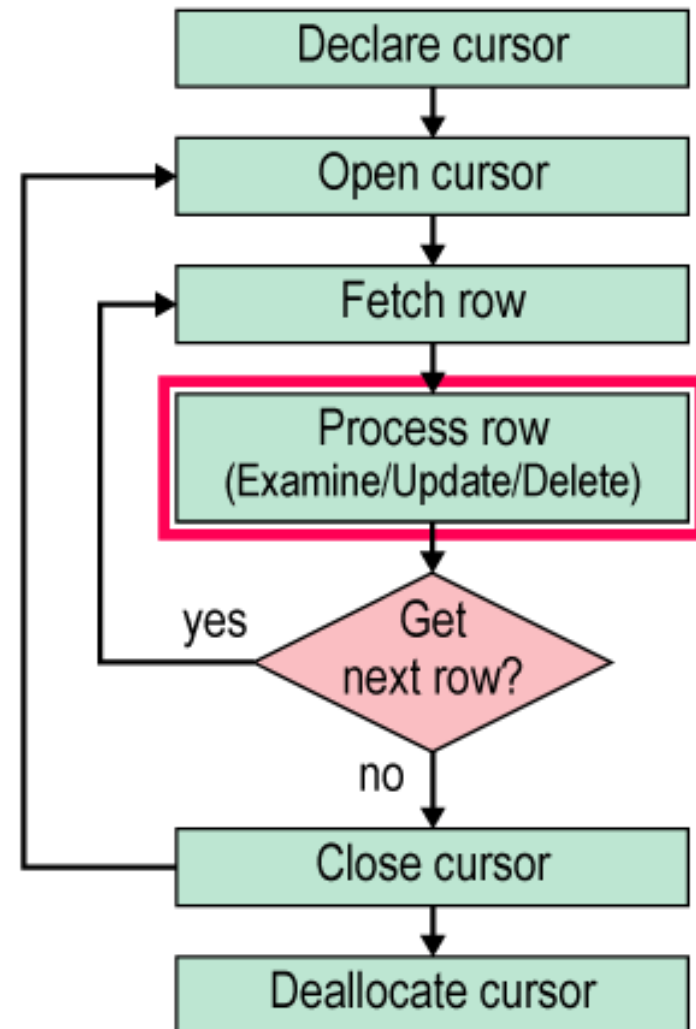
Understanding Cursors

Cursors extend processing of result sets

- Allow positioning at specific rows
- Retrieve one or more rows from the current position
- Support data modification
- Support different levels of visibility
- Provide T-SQL statements access to data

Cursor Life Cycle

1. Declare the cursor
2. Open the cursor
3. Fetch each row
4. Close the cursor
5. Deallocate the cursor



Declare the Cursor

 When you declare a cursor:

 You specify the query

 You specify the cursor mode

● **for read only**

● **for update**

 Adaptive Server reserves memory resources for the cursor

 Update Cursors is the Default Mode

Global Variables for Cursors

@@rowcount

-  After a **fetch**, returns the total number of rows returned by all **fetch** statements for that cursor
-  After any DML statement in a cursor, returns the number of rows affected by that statement

@@sqlstatus

-  Returns 0 if the previous **fetch** was successful
-  Returns 1 for an error resulting from a **fetch**
-  Returns 2 if the previous **fetch** failed because there are no more rows to **fetch**

Alternatives to Cursors

- 📖 Cursors are not always the only way to accomplish a given task

- 📖 Cursor Code Example 1 rewritten using the **case** statement:

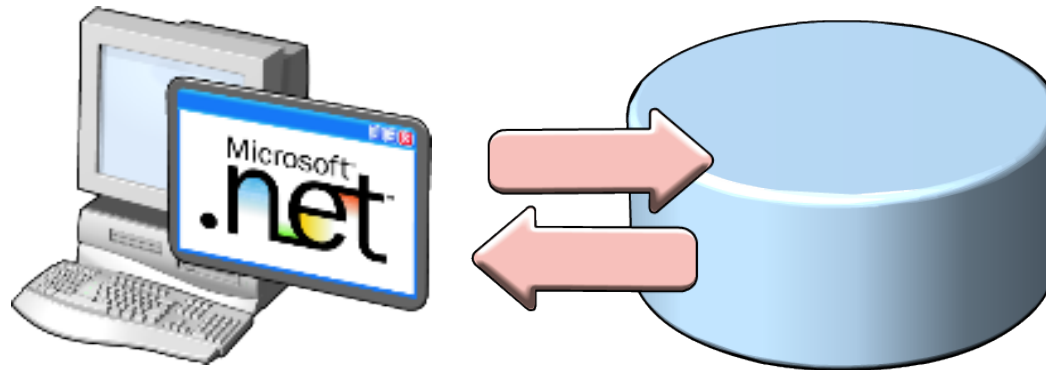
```
select title_id, type,  
       case type  
         when "business" then price * $1.08  
         when "mod_cook" then price  
       end  
from titles  
where type in ("business", "mod_cook")
```

- 📖 Cursor Code Example 1 rewritten using two queries:

```
select title_id, type, price * $1.08  
from titles  
where type = "business"  
select title_id, type, price  
from titles  
where type = "mod_cook"
```

CLR Integration , Assemblies

- Introduced in SQL Server 2005
- Deploy custom developed CLR assemblies within the SQL Server Process
- Leverage the .NET 2.0 API



Implementing Custom Aggregate Functions

- Custom programs in an assembly that are imported into SQL Server® using the integrated CLR
- Can be created using any .NET language such as C# and VB

```
CREATE ASSEMBLY StringUtilities FROM  
'PathToAssembly\StringUtilities.dll'
```

```
WITH PERMISSION_SET=SAFE;
```

```
CREATE AGGREGATE Concatenate(@input nvarchar(4000))  
RETURNS nvarchar(4000)  
EXTERNAL NAME [StringUtilities].[Concatenate]
```